tool-latency-bench

TECHNICAL REPORT · NOVEMBER 2026

How much latency do tool definitions add? A paired benchmark for voice agents, with first results on Gemini

Pradnesh Fernandez A · github.com/pradneshfernandez/tool-latency-bench

ABSTRACT

Voice agents send their full list of tool (function) definitions with every request, and they have roughly one second per conversational turn. We built *tool-latency-bench*, an open benchmark that measures how much time those definitions add. It uses 449 real tool definitions extracted from seven open-source agent projects, a paired-block design that sends the same user message under every condition in random order, and cluster-robust statistics. Two experiments vary the number of tools (0–60) and their structure at a fixed token budget, each in two modes: `tool_choice: none`, which isolates the cost of reading the tools, and `auto`, where the model may call them. Across ~1,250 streamed requests to Google's Gemini API, reading tools on Gemini 3.1 Flash-Lite was cheap: +7.4 ms per 1,000 tool tokens [−12.5, +27.2], or +3.9% [−6.4, +15.3] for 60 tools on an 820 ms baseline. Letting the model call tools cost more (+27.2 ms per 1k [+10.2, +44.2]; +16.5% at 60 tools), mostly the time to emit the call. Tool structure made no detectable difference. A different model (Gemini 3.8 Flash, ~2.2 s) moved latency about 40× more than 60 tools did. We also report that `reasoning_effort: none` does not reliably disable thinking on 3.8 Flash when tools are present, and that Flash-Lite sometimes emits a function call when no tools are declared.

Keywords: voice agents, function calling, tool use, LLM latency, time to first token, benchmarking, paired design

1. Introduction

A voice agent turns speech into text, sends the text to a large language model (LLM), and speaks the reply. Users notice delays of more than about a second, so every step in the pipeline has a small latency budget. Most production voice agents also use *tools*: functions such as `check_order_status` or `transfer_to_human` that the model can call. Each tool is described by a name, a description and a JSON schema for its parameters, and the whole list is sent with every request. A bot with 60 tools sends about 5,000 extra input tokens every time the user speaks.

The *Voice AI & Voice Agents* primer [1] notes that adding function definitions to a prompt can increase latency and that latency-oriented evals for this would be useful. Practitioners respond in different ways: some trim tool lists, split agents into stages that each see fewer tools, or move tools into separate sub-agents. These choices have costs of their own, so it helps to know how large the latency penalty actually is. This report describes a benchmark built to measure it and the first results. It asks three questions:

1. **RQ1, tool count.** How does time to the first answer change as the number of tool definitions grows from 0 to 60?
2. **RQ2, tool structure.** At the same total token budget, do tools with richer schemas (more parameters, enums) cost more than simple ones?
3. **RQ3, reading vs calling.** How much of the cost comes from reading the definitions, and how much from the model choosing to call a tool?

The contributions are: (i) a corpus of 449 real tool definitions with provenance, plus the exact form in which four provider APIs receive them; (ii) an open, reproducible benchmark harness with a paired design and

cluster-robust estimators; (iii) first results on Gemini, with all intervals and checks reported; and (iv) two provider quirks relevant to anyone measuring latency.

2. Background and resources

2.1 Why tool definitions might add latency

An LLM processes all input tokens (the *prefill*) before it can produce the first output token. Tool definitions are input tokens, so in principle more tools mean a longer prefill. Whether this matters in practice depends on how fast the provider's hardware prefills relative to its fixed overheads (network, queueing, scheduling), and on whether the provider caches repeated prompt prefixes. When the model decides to call a tool, the response also has to contain the full call (the function name and JSON arguments), which takes time to generate.

2.2 Resources this work builds on

- **Voice AI & Voice Agents: An Illustrated Primer** [1] (CC0): the motivation, from its section on function calling.
- **Pipecat** [2] (BSD 2-Clause): an open-source voice agent framework. Tool formats and the function-calling flow follow its examples and adapters, and its examples are a source of tools.
- **aiewf-eval** [3] (MIT): a voice-agent evaluation. The project layout, CLI and the definition of TTFAT (time to first answer token) follow it.
- **pipecat-ai/stt-benchmark** [4]: conventions for latency reporting and the contributor workflow.
- **A statistical approach to model evaluations** [5] (Anthropic, 2024): the paired-difference, clustered-standard-error and power analysis approach used in Section 5.
- **Tool sources** [6–11]: τ-bench, τ²-bench, EVA, Pipecat Flows and LiveKit Agents, listed in Table 1.

3. Tool corpus

3.1 Extraction

Synthetic tools would make the results easy to dismiss, so the benchmark uses real definitions. 660 tool definitions were extracted from seven open-source projects at pinned commits, each with the project's own schema code (for example, Pydantic models or decorators), keeping the wording exactly as in the source. Each file records the source repository, commit and path. A rebuild script re-clones the sources and checks the pool matches byte for byte. Package versions are pinned because the Pydantic version changes the generated schemas.

Table 1. Sources of the tool corpus (after deduplication).

Source	Repository	Commit	License	Tools
τ-bench	sierra-research/tau-bench	`59a200c`	MIT	30
τ²-bench	sierra-research/tau2-bench	`989d328`	MIT	67
aiewf-eval	kwindla/aiewf-eval	`ff1e9fa`	MIT (README)	5
EVA	ServiceNow/eva	`2918ca6`	MIT	122
Pipecat (examples)	pipecat-ai/pipecat	`5558d26`	BSD 2-Clause	70
Pipecat Flows (examples)	pipecat-ai/pipecat-flows	`96223f4`	BSD 2-Clause	49
LiveKit Agents (examples)	livekit/agents	`5b49888`	Apache 2.0	106
Total				**449**

3.2 Deduplication

Only exact duplicates (same name, description and parameters) were removed, by fixed rules, leaving 449 of 660. Most came from Pipecat's per-provider examples, which copy the same tools (−175); Pipecat's bundled copies of the Pipecat Flows examples were kept only under Pipecat Flows (−35); and one Pipecat Flows tool identical to Pipecat's weather demo was removed (−1). Tools that share a name but differ in content are kept.

3.3 Definitions as sent

Frameworks rewrite tool definitions before sending them, differently for each provider. LiveKit's OpenAI and Anthropic paths use strict mode (titles removed, `additionalProperties: false`, all properties required); litellm inlines `$defs` for Gemini; the google-genai SDK used by Pipecat uppercases types. To record this, each framework's real request was captured at a local HTTP server (no provider contacted) for four request formats (OpenAI Chat, OpenAI Responses, Anthropic, Gemini). The benchmark itself sends the pool definitions in the OpenAI Chat `tools` format.

3.4 Grouping and token counts

Each tool is put in one of three groups by its parameter schema: **complex** if any parameter contains an object, an array or a `$ref`; otherwise **medium** if it has three or more parameters or any enum; otherwise **simple**. Tokens are counted with tiktoken (`o200k_base`) on the compact JSON definition.

Table 2. Tool groups in the corpus and the pools drawn from them.

Group / pool	Rule or selection	Tools	Median tokens	Range
simple	≤2 flat parameters, no enum	308	80	
medium	≥3 parameters or an enum, flat	113	194	
complex	nested object / array / $ref	28	266	
exp1 pool	simple tools in the narrowest token range holding ≥150	151	75	52–93
exp2 simple	simple tools near the per-tool budget target	47	151	125–179
exp2 medium	medium tools near the per-tool budget target	40	157.5	125–185

Complex tools are too few and too varied in size to support a fixed-budget comparison, so they are not used in the experiments.

4. Experimental design

4.1 The request

Every request has the same shape: a system prompt for a customer-service voice agent of an online retailer, no conversation history, one of 30 short user messages (Appendix A), and a set of tools. Settings are fixed for the whole campaign (Table 3). A random 32-character nonce is put at the start of the system prompt of every request, and tool order is random, so no two requests share a prefix that a provider could cache. If a request has no tools, `tool_choice` and `parallel_tool_calls` are left out because some APIs reject them.

Table 3. Fixed request settings.

Setting	Value	Reason
temperature	`0`	less output variation
max_tokens	`100`	voice replies are short; caps total time
stream	`true, include_usage`	timing of first token; provider token counts
parallel_tool_calls	`false`	one call per response
tool_choice	`none | auto`	the two modes (Section 4.4)
reasoning_effort	`lowest accepted`	thinking adds its own latency

4.2 Experiment 1: tool count

Conditions are 0, 10, 30 and 60 tools from the exp1 pool, whose tools have similar sizes (52–93 tokens) so that count and tokens grow together evenly. In each block, 60 tools with distinct names are drawn, 30 of those are chosen, and 10 of the 30. The subsets are nested, so a larger condition always contains the smaller one, which reduces noise from which tools happen to be drawn. Provider-counted tool load was about 810, 2,440 and 4,870 tokens. The pool has 151 tools but only 121 distinct names; since a request never repeats a name, 121 is the most tools a single request can hold.

4.3 Experiment 2: tool structure at a fixed budget

Conditions are no tools, 20 simple tools, 20 medium tools, and 10 simple plus 10 medium (mixed). All three tool sets are drawn so their tiktoken total is within ±5% of a 3,060-token budget, by rejection sampling. This holds count and size fixed and varies only how the tokens are spent: on long descriptions (simple) or on parameters and enums (medium). To meet the budget the simple tools come from the top of their size range (125–179 tokens), so in practice the contrast is "description-heavy vs schema-heavy" tools of about 150 tokens each.

4.4 Two modes

- **`tool_choice: none` (primary).** The tools are in the prompt but the model must answer in text. The time to the first answer text therefore isolates the cost of reading the definitions. Provider input-token counts confirm the tools stay in the prompt in this mode.
- **`tool_choice: auto` (secondary, realistic).** The model may call a tool. The latency then also includes generating the call, and whether it calls one depends on whether the drawn set contains a relevant tool.

4.5 Metrics

The primary metric, `actionable_ms`, is the time from sending the request to the first non-whitespace answer text, or to the completed tool call if a tool call starts first. It is the moment a voice agent can act: start speaking, or run the tool. A secondary metric, `ttfat_ms`, takes the first answer text or the first tool-call chunk; it is not compared across providers because some send a whole call in one chunk. A leading `<think>…</think>` block in the content is skipped. Each request also records the provider's input, output, reasoning and cached token counts and the finish reason.

4.6 Blocks, order and pacing

A *block* sends one user message once under every condition, in a random order drawn per block. Thirty blocks make a session, so each of the 30 user messages is used once. Each run starts with three warm-up requests (discarded), waits 1 s between requests, retries HTTP 429 with backoff up to six tries, and never runs at the same time as another run, since concurrent requests would affect each other's latency.

5. Statistical analysis

The analysis follows [5] and is implemented in the repository's `analysis` module.

- **Paired log-ratios.** For each block and condition *c*, the effect is the difference in log latency from the same block's 0-tool request. The mean is reported as a percentage change, `100·(exp(mean) − 1)`. Pairing cancels most network and load variation, which is shared within a block.
- **Clustered standard errors and t intervals.** Standard errors are clustered on the user message. 95% intervals use the t distribution with (clusters − 1) degrees of freedom.
- **Dose-response slope (exp1).** A per-block regression of latency on provider tool tokens gives an added latency in ms per 1,000 tool tokens, and in percent per 1,000.
- **Provider tokens.** Providers render tool schemas differently, so tiktoken is only an approximation. Tool load is measured as the provider's input tokens minus the same block's 0-tool request, and rates use those tokens.
- **Direct contrasts (exp2).** Medium vs simple and mixed vs simple are estimated as paired contrasts within each block, both as ratios and as a difference in ms per 1,000 provider tokens. A warning is raised when contrasted sets differ by more than 5% in provider tokens.
- **Robust estimates and power.** Medians with cluster-bootstrap intervals are reported next to the means. Each effect also reports the smallest change detectable with 80% power, and the number of blocks needed to detect 5%.
- **Errors.** Blocks with an errored request are dropped, and errors are counted per condition, since uneven errors can bias results.
- **Pooling.** The repository's `compare` command shows sessions side by side. This report also gives pooled estimates: the complete blocks of both sessions analyzed together, still clustered on the user message.

6. Procedure

6.1 Models and settings

All requests went to Google's Gemini API through its OpenAI-compatible endpoint on the paid tier. Before the campaign, the lowest accepted reasoning setting was checked for each model, along with whether it actually turned thinking off:

- **gemini-3.1-flash-lite** (main model): `reasoning_effort: none` accepted; 0 reasoning tokens with and without tools.
- **gemini-3.8-flash**: `minimal` returned HTTP 400; `none` was accepted but the model still thinks at times when tools are present (Section 7.4).

6.2 Campaign

Every new model got a two-block smoke run first, checking no errors, input tokens growing with tool count in both modes, and zero cached tokens; the smoke folder was then deleted. Full runs were done strictly one after another on 6 November 2026, 16:34–18:29 IST: for Flash-Lite, exp2 and exp1 in both modes (session s1), then both again (session s2); for 3.8 Flash, exp1 in both modes once. The request parts, pools and token statistics were not changed between sessions. A caching probe and a diagnostic probe of the hallucinated tool call followed (Section 8).

Table 4. Requests sent.

Part	Runs	Requests	Errors
Flash-Lite, exp1 + exp2, both modes, 2 sessions	8 × 30 blocks	960	0–1 per run (empty stream); no 429/503
3.8 Flash, exp1, both modes	2 × 30 blocks	240	2 (none), 20 (auto) empty
Caching probe (120 tools)	25 rounds	53	0
Total		**~1,250**	

7. Results

7.1 RQ1: reading tools is cheap

With `tool_choice: none`, latency barely moved as tools were added (Figure 1, Table 5). The pooled slope was **+7.4 ms per 1,000 tool tokens [−12.5, +27.2]** (+0.9% per 1k [−1.0, +2.9]). At 60 tools (~4,875 provider tokens) the change was **+3.9% [−6.4, +15.3]**. Read as an upper bound, 60 tools add at most ~130 ms; the point estimate is ~35 ms on an 820 ms baseline. The two sessions agree within noise (Figure 2): s1 +13.7 [−10.8, +38.2], s2 +1.1 [−25.6, +27.7] ms per 1k.

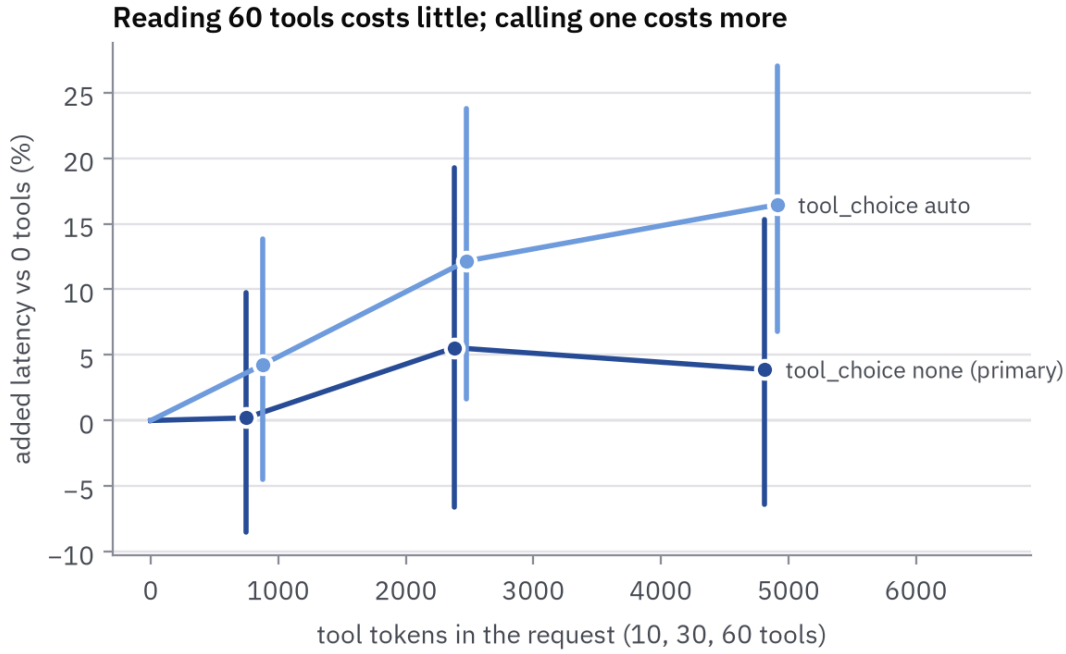


Figure 1. Experiment 1, Gemini 3.1 Flash-Lite, two sessions pooled. Mean change in `actionable_ms` against the same block's 0-tool request, by provider tool tokens (10, 30, 60 tools), for both modes. Bars: 95% intervals.

Table 5. Experiment 1, Flash-Lite, pooled (60 blocks per mode).

Tools	Tool tokens	Median (none)	vs 0 (none)	Median (auto)	vs 0 (auto)	Calls (auto)
0	0	820 ms	–	895 ms	–	3%
10	~810	840 ms	+0.2% [−8.5, +9.7]	910 ms	+4.3% [−4.5, +13.8]	58%
30	~2,430	878 ms	+5.6% [−6.6, +19.2]	923 ms	+12.2% [+1.6, +23.8]	87%
60	~4,860	837 ms	+3.9% [−6.4, +15.3]	1,080 ms	+16.5% [+6.8, +27.0]	93%
Slope per 1,000 tool tokens			+7.4 ms [−12.5, +27.2]		+27.2 ms [+10.2, +44.2]	

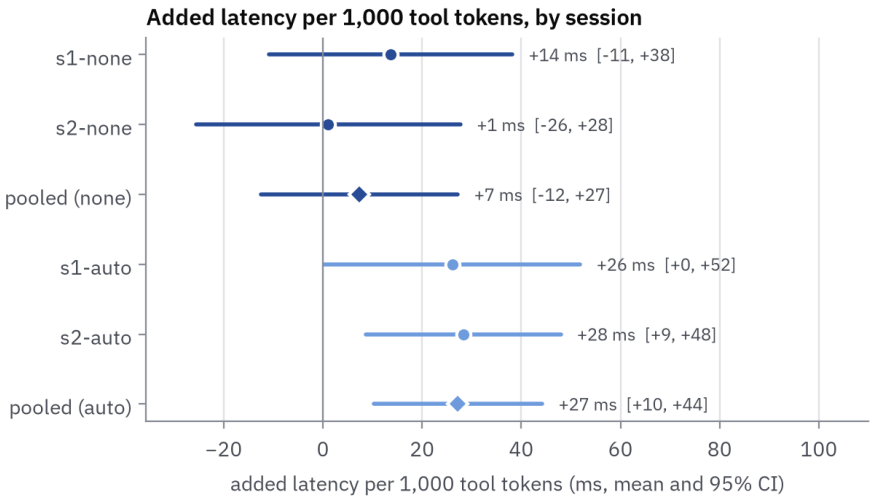


Figure 2. Slope in ms per 1,000 tool tokens for each session and pooled, both modes, with 95% intervals.

7.2 RQ3: calling tools costs more

With `tool_choice: auto` the pooled slope was **+27.2 ms per 1,000 tool tokens [+10.2, +44.2]** (+3.2% per 1k [+1.4, +4.9]), and 60 tools added **+16.5% [+6.8, +27.0]** (median 895 → 1,080 ms). Both sessions agree (+26.1 and +28.3). The model called a tool in 58%, 87% and 93% of requests at 10, 30 and 60 tools. At 60 tools, responses with a tool call had a median of 1,082 ms (n = 56), against 876 ms for text answers (n = 3, too few to rely on). Since reading costs only ~7 ms per 1k, most of the difference between modes appears to be the time to generate a complete call. For a voice agent this is a real cost, because it cannot act until the call is complete, but it is a different cost from reading the tools.

7.3 RQ2: tool structure makes no detectable difference

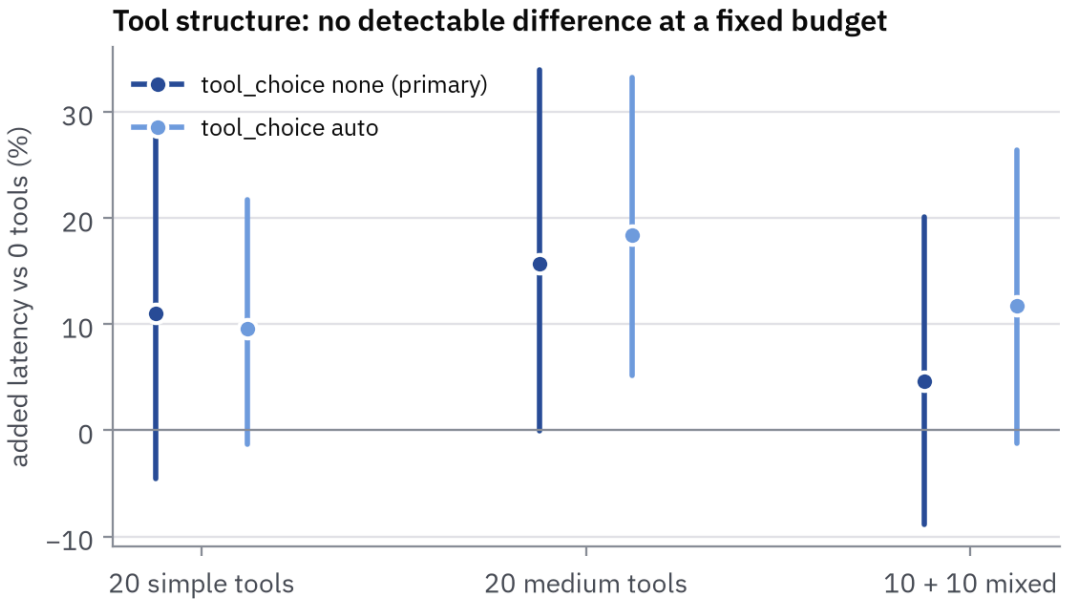


Figure 3. Experiment 2, Flash-Lite, two sessions pooled. Change against no tools for 20 simple, 20 medium and 10 + 10 mixed tools at the same ~3,060 tiktoken budget, both modes, with 95% intervals.

Table 6. Experiment 2, Flash-Lite, pooled.

Condition	Provider tokens	vs none (none)	vs none (auto)	Tool-call rate (auto)
20 simple	~3,350	+11.0% [−4.5, +29.0]	+9.6% [−1.3, +21.7]	57%
20 medium	~3,810	+15.7% [−0.1, +34.0]	+18.4% [+5.1, +33.3]	42%
10 + 10 mixed	~3,575	+4.7% [−8.8, +20.1]	+11.7% [−1.2, +26.4]	48%
Medium vs simple	+13.8%	+4.3% [−8.1, +18.3]	+8.0% [−1.4, +18.3]	
per 1k tokens, difference		+9.9 ms [−48.7, +68.4]	+27.2 ms [−16.0, +70.4]	
Mixed vs simple	+6.9%	−5.7% [−14.8, +4.3]	+1.9% [−8.3, +13.4]	

The direct contrast, medium vs simple in the primary mode, was **+4.3% [−8.1, +18.3]**, with a per-token rate difference of **+9.9 ms per 1k [−48.7, +68.4]**. The sessions disagree on the sign (s1 +17.9%, s2 −8.2%), as expected for an effect near zero. The budget was matched in tiktoken tokens, but Gemini counts the medium sets as **13.8%** more tokens than the simple ones (mixed 6.9%). The per-token difference corrects for this; the plain ratios do not. The exp2 results are consistent with exp1: 20 simple tools (~3,350 tokens) +11.0% here, against +5.6% for 30 tools and +3.9% for 60 tools in exp1, with overlapping intervals.

7.4 The model matters far more than the tools

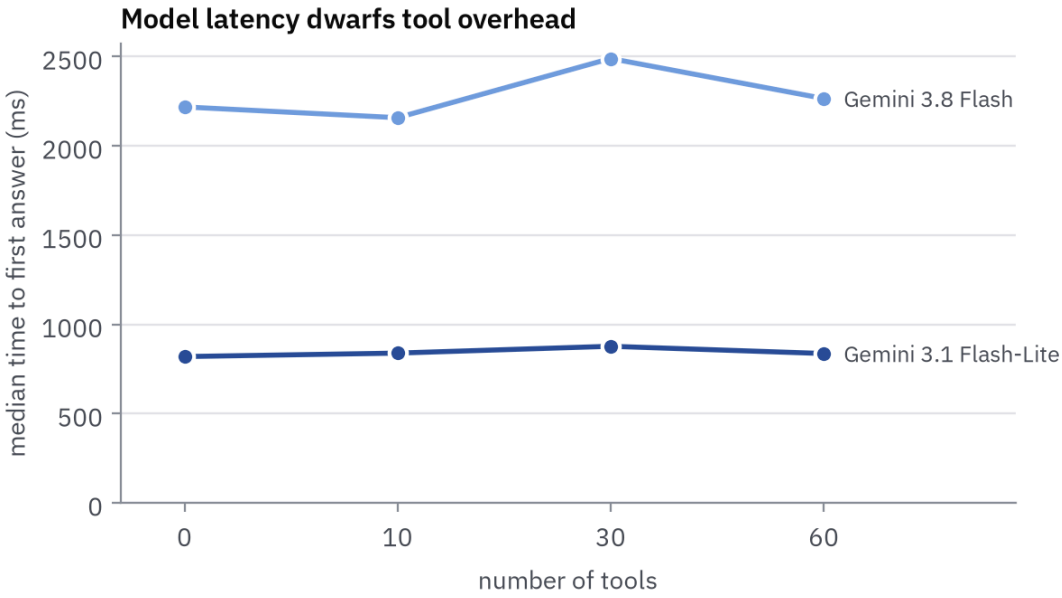


Figure 4. Median `actionable_ms` by tool count for Gemini 3.1 Flash-Lite (two sessions pooled) and Gemini 3.8 Flash (one session), `tool_choice: none`.

Gemini 3.8 Flash had a median of **~2.2 s** at every tool count (2,218 / 2,158 / 2,489 / 2,265 ms at 0 / 10 / 30 / 60 tools), against 0.82 s for Flash-Lite. The gap between the models (~1.4 s) is about 40× the estimated cost of 60 tools. 3.8 Flash could not be measured cleanly on this endpoint: with tools present, `reasoning_effort: none` does not reliably stop it from thinking. It sometimes spent ~95 hidden tokens thinking, reached `max_tokens: 100` and returned `finish_reason: length` with no visible output (2 of 120 requests with `none`, 20 of 120 with `auto`). Its latencies therefore mix thinking time with tool reading, so they are reported as a finding about the model, not as a tool-overhead estimate.

8. Robustness checks

8.1 Prompt caching

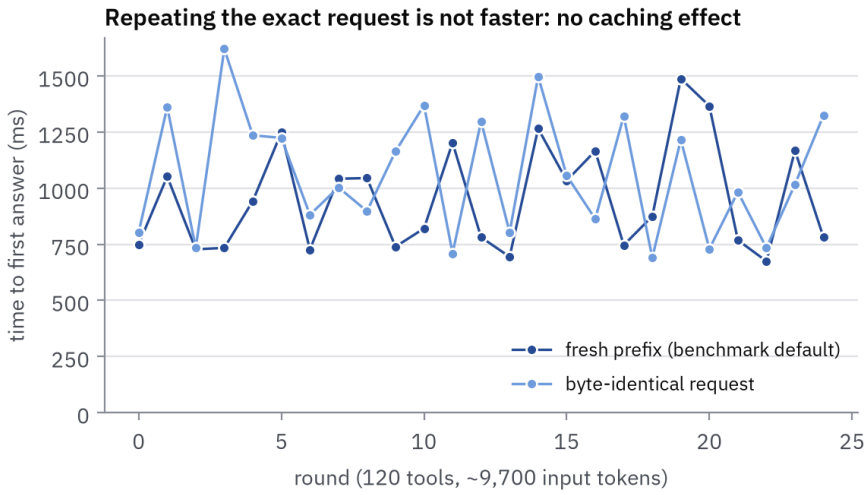


Figure 5. Caching probe. 25 rounds; each round sends a byte-identical 120-tool request (~9,700 input tokens) and the same request with a fresh nonce, in random order.

Gemini's endpoint reports no `cached_tokens`, so the nonce's effect cannot be checked from usage data. A separate probe compared a request repeated byte for byte with the same request under a fresh nonce. The ratio identical / fresh was **1.11 [0.96, 1.29]**: repeating the request was not faster, so there is no sign that caching lowered latency in the benchmark. The fresh 120-tool median (875 ms) was also close to the 0-tool baseline, in line with RQ1.

8.2 Hallucinated tool calls

About 3% of responses in `none` mode were tool calls, including with no tools at all. All came from one message, "How many reward points do I have right now?". A diagnostic probe showed Flash-Lite returns an invented function call (`get_reward_points`, `get_customer_reward_points`) with no tools declared in 3 of 6 retries, and ignores `tool_choice: none` for it. Removing that message leaves the results unchanged: the `none` slope stays +7.4, and the `auto` slope goes from +27.2 to +29.5 ms per 1k.

8.3 Order, drift, noise and power

- **Position in block.** Medians by position were 882 / 877 / 845 / 789 ms (exp1, `none`): the first request of a block is slower. Because conditions are shuffled per block, this adds noise but no bias.
- **Drift.** Median of the first 10 vs the last 10 blocks: 824 vs 853 ms. Negligible.
- **Session differences.** Baselines differed between sessions (exp1 `auto`: 984 vs 740 ms at 0 tools), which pairing removes.
- **Noise and power.** The coefficient of variation is ~0.3, with p10–p90 about 690–1,290 ms at every tool count. One 30-block session detects changes of about ±20%; detecting 5% would take ~400 blocks. This is why sessions are pooled, and why a "no difference" result here rules out large effects, not small ones.

9. Discussion

On a fast model behind a production API, a few thousand tokens of tool definitions are a small share of time to first token. The fixed parts of a request (network, queueing, scheduling, and producing the first output) dominate. This fits modern serving, where prefill is highly parallel. It means that for Flash-Lite-class models, trimming a tool list from 60 to 10 saves at most around a hundred milliseconds, and probably a few tens.

The cost that does show up is the tool call itself. When the model calls a tool, the agent waits for the full call before it can act, and with more tools the model calls one more often. Designs that cut the time to act (shorter argument schemas, filler speech while a tool runs, or answering directly when no tool is needed) target the larger cost. Choosing the model and keeping reasoning off matter more still: switching models changed latency by over a second.

These conclusions are for one provider. Providers with different hardware or serving stacks may behave differently in either direction, which is the main reason for the next steps below.

> **Practical takeaway.** On a fast model, don't trim your tool list to save latency. Pick the model, keep reasoning off, and design around tool calls.

10. Limitations

- **One provider, one main model, one afternoon.** Both sessions ran within two hours; time-of-day and day-to-day effects are untested.
- **Compatibility endpoint.** Requests used Gemini's OpenAI-compatible endpoint, not the native API that Pipecat's `GoogleLLMService` uses, and not the framework-rewritten definitions of Section 3.3.
- **Count and tokens move together** in exp1, so it measures tool-definition load and cannot separate "more tools" from "more tokens".
- **exp2 budget in tiktoken tokens**, not the provider's; the per-token contrast corrects for this.
- **Short prompts and outputs.** ~100 tokens before the tools, no history, output capped at 100 tokens. Long conversations may behave differently.
- **At most 60 tools** in the benchmark runs (120 only in the caching probe).
- **Precision.** The intervals exclude large effects but cannot distinguish 0 from a few percent.

11. Future work

1. More Flash-Lite sessions on other days and times, pooled.
2. OpenAI as the second provider; a fast-inference provider (Groq, Cerebras), where overhead may be close to zero.
3. Experiment 3: 0 / 30 / 60 / 90 / 120 tools from the existing pool, to test whether growth stays linear.
4. Native Gemini API vs the OpenAI-compatible endpoint, interleaved within blocks; and whether `thinkingBudget: 0` makes 3.8 Flash measurable.
5. Tool-selection accuracy as the list grows: "N tools cost X ms and Y% wrong picks".
6. An exp2 variant with the budget matched in provider tokens.
7. A reproducible report to Google on the hallucinated tool call, and hallucinated calls per model as a side metric.

12. Reproducibility

The harness, tool corpus, pools, request parts and analysis code are in the repository under the MIT license (tool definitions keep their original licenses). Each run folder records the results, the model and settings, the random seed, package versions and the exact request parts used; API keys are never written. A full campaign of both experiments in both modes takes about 20 minutes and about 1.3 million input tokens.

```
uv sync
uv run tool-latency-bench run exp1_tool_count --model gemini-3.1-flash-lite --service google \
    --session s1 --tool-choice none --reasoning-effort none
uv run tool-latency-bench analyze runs/exp1_tool_count/<run folder>
uv run tool-latency-bench compare exp1_tool_count
```

The repository also includes a ready-made Claude Code prompt and a `CLAUDE.md` with the run rules (smoke run first, one run at a time, both modes, lowest reasoning setting, stop on 429/503). Results from other providers are welcome as pull requests.

Acknowledgements

Thanks to the authors of the Voice AI & Voice Agents primer, Pipecat and aiewf-eval, and to the maintainers of the projects whose tool definitions make up the corpus.

References

1. Voice AI & Voice Agents: An Illustrated Primer. voiceaiandvoiceagents.com (source: github.com/pipecat-ai/voice-ai-primer-web), CC0.
2. Pipecat: open-source framework for voice and multimodal agents. github.com/pipecat-ai/pipecat, BSD 2-Clause.
3. aiewf-eval: voice-agent evaluation. github.com/kwindla/aiewf-eval, MIT.
4. pipecat-ai/stt-benchmark: speech-to-text latency benchmark. github.com/pipecat-ai/stt-benchmark.
5. E. Miller. A statistical approach to model evaluations. Anthropic, 2024. anthropic.com/research/statistical-approach-to-model-evals.
6. τ-bench: tool-agent-user interaction benchmark. github.com/sierra-research/tau-bench, MIT.
7. τ²-bench. github.com/sierra-research/tau2-bench, MIT.
8. EVA. github.com/ServiceNow/eva, MIT.
9. Pipecat Flows. github.com/pipecat-ai/pipecat-flows, BSD 2-Clause.
10. LiveKit Agents. github.com/livekit/agents, Apache 2.0.
11. tiktoken (o200k_base encoding). github.com/openai/tiktoken.

Appendix A. User messages

The 30 user messages, one per block in each session. Each is sent once per condition.

id	Text	id	Text
order_status	Hi, I placed an order last week and it still hasn't arrived. Can you check where it is?	return_item	I'd like to return a jacket I bought, it doesn't fit.
update_address	Can you change the shipping address on my account? I just moved.	billing_charge	I think I was charged twice for the same purchase.
store_hours	What time do you close today?	cancel_order	I need to cancel the order I placed this morning.
refund_status	I returned something two weeks ago and haven't seen the refund yet.	reset_password	I can't log in to my account, I think I forgot my password.
change_size	Can I swap the shoes I ordered for a size nine instead?	damaged_item	My package arrived today and the lamp inside is broken.
wrong_item	You sent me the wrong color, I ordered the blue one.	delivery_date	When will my new couch be delivered?
talk_to_human	Can I please speak to a real person?	update_payment	I want to update the credit card on file for my subscription.
promo_code	My discount code isn't working at checkout.	stock_check	Do you have the gray hoodie in a large in stock?
warranty	Is my coffee machine still under warranty? I bought it last year.	missing_item	My order came but one of the three items is missing.
change_email	I'd like to change the email address on my account.	cancel_subscription	How do I cancel my monthly subscription?
gift_card_balance	Can you tell me how much is left on my gift card?	price_match	I saw this same TV cheaper on another site, do you price match?
late_delivery	The tracking says delivered but I never got anything.	invoice_copy	Could you send me a copy of the invoice for my last order?
store_location	Is there a store near downtown I can visit?	exchange_policy	What's your policy on exchanges after thirty days?
add_to_order	Can I add another item to an order I already placed?	loyalty_points	How many reward points do I have right now?
book_appointment	I'd like to book an appointment for a fitting this Saturday.	complaint	I'm really unhappy with how my last return was handled.

Appendix B. Per-session results

Table B1. Median `actionable_ms` by session and condition (30 blocks each), and errors per run.

Experiment · model · mode · session	Cond. 1	Cond. 2	Cond. 3	Cond. 4	Errors
exp1 (0 / 10 / 30 / 60 tools)					
Flash-Lite · none · s1	849	753	853	845	0
Flash-Lite · none · s2	779	998	949	807	0
Flash-Lite · auto · s1	984	947	1,077	1,107	1
Flash-Lite · auto · s2	740	855	882	968	1
3.8 Flash · none · s1	2,218	2,158	2,489	2,265	2
3.8 Flash · auto · s1	2,371	2,434	2,439	2,314	20
exp2 (none / simple / medium / mixed)					
Flash-Lite · none · s1	923	1,092	1,179	996	0
Flash-Lite · none · s2	827	918	956	922	1
Flash-Lite · auto · s1	805	994	1,083	923	1
Flash-Lite · auto · s2	966	990	1,019	1,077	0

Table B2. exp1 slope per 1,000 provider tool tokens, by session.

Mode	s1	s2	Pooled
none	+13.7 [−10.8, +38.2]	+1.1 [−25.6, +27.7]	+7.4 [−12.5, +27.2]
auto	+26.1	+28.3	+27.2 [+10.2, +44.2]